PORTFOLIO · HBM DIGITAL DESIGN (RTL DESIGN)

Mini DRAM Controller

Command-level FR-FCFS · 행 / 열 명령 버스 · Per-bank Refresh · RFM — Spec · RTL · 검증 · PPA

지원 직무	SK hynix · HBM Digital Design (RTL Design)
지원자	권용현 · 010-6408-8646 · kyongh0331@naver.com
저장소	`github.com/yhkwon-git/mini-dram-controller` · `v4 commit ab9d522` · `RTL 44a0bb3`
사용 도구	SystemVerilog · Verilator 5.052 · Yosys 0.52 · sv2v · OpenSTA 3.1 (Nangate45) · Python

p.2–26의 v4 수치는 RTL commit ab9d522 기준(44a0bb3은 ModelSim 캐스트와 q_empty 출력만 더했고 directed 로그가 같다). p.12–13 파형은 44a0bb3의 VCD, 버전 비교는 각 버전 commit 재실행, 분석 스크립트 322632f, HSL 숫자는 HSL e61e90d.

00 Summary

v0 → v4: 단일 FSM에서 행 / 열 명령 버스 · Per-bank Refresh · RFM까지

동일 DRAM 모델 기반 컨트롤러 5단계 설계 · 버전 전환마다 latency · 처리량 · 면적 · 타이밍을 동일 TB · 동일 합성 Flow로 비교

버전	구조 및 변경 사항	포화 row hit RD	fmax · 면적
v0	단일 FSM: 요청 하나를 PRE · ACT · RD/WR 순서로 처리	–	1,182 MHz · 1,208 μm²
v1	깊이 4 큐 + 뱅크 엔진 ×4 + 사이클당 명령 1개 arbiter	19.9%	343 MHz · 9,692 μm²
v2	명령 단위 FR-FCFS(age matrix), tREFI refresh + PREA, RD 4개 파이프라인, 타이밍 7종 추가	66.7%	357 MHz · 13,478 μm²
v3	읽기 · 쓰기 큐 8 + 8, posted write · 포워딩, 쓰기 묶음, page 정책 3 · 주소 매핑 4	66.7%	337 MHz · 18,600 μm²
v4	행 · 열 명령 버스, 뱅크 그룹(tCCD · tRRD · tWTR S/L), per-bank refresh, RFM	**80.7%**	**348 MHz** · 18,912 μm²

포화: 요청 간격 0, back-pressure 없음, seed 1, 5,000건. v1 처리량은 2eabd33, 면적 · 타이밍은 fa08330. v4부터 장치에 뱅크 그룹이 있어 동일 장치의 v3 방식 빌드(bus1_refab)는 63.3%. 면적 · fmax는 Nangate45 typical, mem_ctrl 단독.

80.7%

포화 row hit 읽기, 데이터 버스 사용률 · 동일 장치의 v3 방식 63.3% (+27%)

1,200 / 1,200

랜덤 회귀 PASS · 600만 요청 · 빌드 16 · 규칙 위반 0 · 기능 bin 32/32

98.7 · 98.2%

컨트롤러 Line · Branch 커버리지 (Toggle 93.1%), directed + 랜덤

348 MHz

Nangate45 최소 주기 2.874 ns · 18,912 μm² · FF 1,616 (v3 337 MHz)

최대 성능 요인: per-bank refresh

all-bank로 바꾸면 포화 평균 −16%, 면적 차이는 2%. 버스 분리와 함께일 때 row hit 63.3 → 80.7%

타이밍 클로저: 208 → 337 MHz (v3)

수락 경로를 스케줄러에서 분리, 후보 집합 통합, one-hot 선택 유지. v4는 기능 추가 후에도 348 MHz

검증: 독립 체커 + 랜덤 회귀

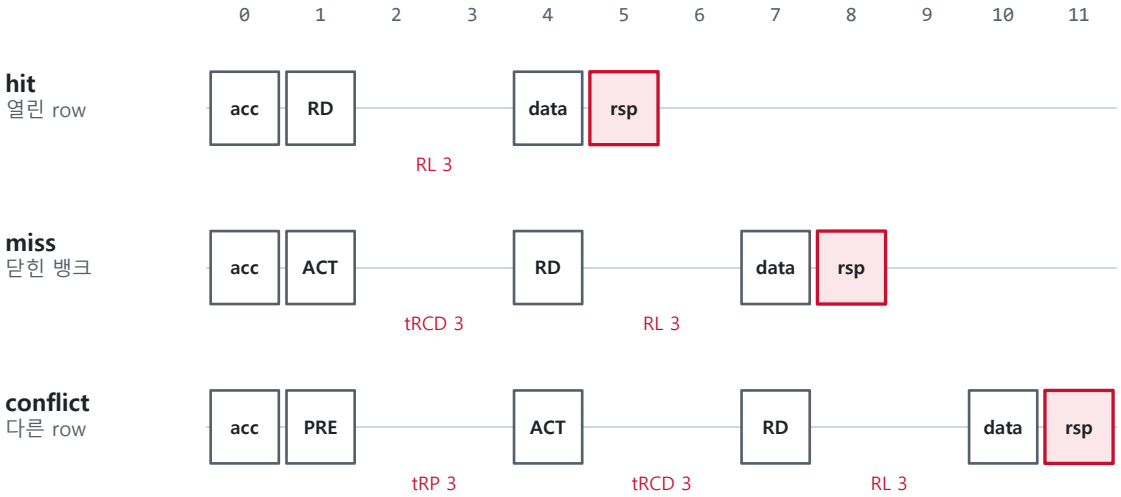
DRAM 핀만 관측하는 bind 체커(타이밍 18 · 프로토콜 10종)와 class 랜덤 TB. 회귀 1,200회 위반 0, TB race 1건 디버그

문서 구성 p.3–21 v4 상세 (Spec, 구조, RTL, 파형, 검증, 디버그, trade-off, HSL 통합, 버전별 latency) → p.22–26 v2 – v4 비교 (구조, 검증 인프라, 성능, 트래픽 × 기능 분석, PPA)

Controller 역할: Bank 대기 시간을 타 요청 처리로 은닉 (Latency Hiding)

동일 뱅크에서도 page hit RD 5사이클, row conflict RD 11사이클 · 타이밍은 자체 축소 모델(JEDEC 값 아님) · v4, commit ab9d522

요청별 명령 순서 (v4, 수락 엣지 = 0 기준, 이후 엣지 단위)



- **수락 → 첫 명령 1사이클**: 수락 엣지에 큐 입력, 다음 엣지에 스케줄러가 선택한 명령 발행
- **RL 3 후 데이터, 1사이클 후 응답**: 데이터를 응답 버퍼에 저장한 다음 엣지에 rsp_valid. 쓰기와 포워딩된 읽기는 수락 다음 엣지에 응답(1사이클)
- **PRE · ACT 대기 6사이클**로 conflict latency가 hit의 2배 이상. 이 구간에 다른 뱅크의 RD/WR · ACT를 채우는 것이 처리량을 결정

v4 설계 범위

- host_if → mem_ctrl(읽기 큐 8 + 쓰기 큐 8, 스케줄러, 뱅크 ×4, 채널, refresh, 응답) → dram_model(뱅크 그룹 2 × 뱅크 2). 단일 클럭, 비동기 리셋
- DRAM 측: 행 버스(ACT · PRE · PREA · REF · REFpb)와 열 버스(RD · WR) 분리, 고정 지연 RL 3 · WL 1. busy 신호 없이 모든 타이밍 규칙을 컨트롤러가 관리
- Host 측: valid/ready, 응답은 요청 순서(in-order), 쓰기는 posted
- Out of Scope: 실제 PHY, multi-rank, ECC, Sign-off PPA

HBM에서의 위치

- 표준 HBM(HBM3E까지)은 컨트롤러가 Host SoC에 위치. HBM4부터 컨트롤러를 base die에 두는 custom HBM 구조가 등장했고, High-Speed Link Lite가 이 RTL을 해당 위치에 적용(p.20)

구성 (p.3–21)	내용
p.4–6	Spec(주소 · 명령 · 인터페이스 · 타이밍), 요청 처리 흐름과 FF 구성
p.7–13	RTL 블록 5종, v4 파형 2종
p.14–21	검증 · 디버그 · trade-off · HSL 통합 · 버전별 latency

Spec ①: 주소 매핑 · 명령 집합 · DRAM / Host Interface 정의

상수는 rtl/dram_pkg.sv 단일 정의, docs/spec.md에 동일 값 문서화 · 4 bank × 256 row × 16 col × 32 bit (64 KiB) · 14 bit 워드 주소

주소 매핑 (ADDR_MAP, 모두 전단사)

매핑	배치 (MSB → LSB)	효과
XOR (기본)	row | bank ^ row[1:0] | col	16 word 단위 row, 64 word stride도 뱅크 분산
row:bank:col	row | bank | col	16 word 단위 row, 이후 다음 뱅크
row:col:bank	row | col | bank	연속 word가 뱅크 순환
bank:row:col (v2)	bank | row | col	뱅크마다 주소 공간 1/4

명령 (dram_cmd_e, 3 bit)

버스	명령 (코드)	동작
행	ACT 001 · PRE 100 · PREA 110	row open · 단일 뱅크 close · 전체 close
행	REF 101 · REFpb 111	전체 뱅크(이후 tRFC 12) · 단일 뱅크(이후 tRFCpb 8)
열	RD 010 · WR 011 (+ ap)	열린 row에 32 bit 버스트 1회. row 필드 없음, ap 시 close

- **두 버스는 매 엣지 샘플**: 한 엣지에 행 명령 1개와 열 명령 1개 동시 발행 가능(HBM 방식). 열 버스는 해당 뱅크의 열린 row 사용
- **매핑은 TB에서 독립 구현**: tb_loc은 비트 슬라이스 대신 나눗셈 · 나머지로 동일 매핑을 계산해, RTL 매핑 버그를 짝 없는 명령으로 검출

DRAM 측 (DFI 형태, busy 없음, 읽기는 data_valid 순서)

신호	방향	의미
dram_rcmd · rbank · row	출력	이번 사이클의 행 명령
dram_ccmd · cbank · col · ap	출력	이번 사이클의 열 명령
dram_wdata_en · wdata	출력	WR 후 WL(1) 엣지에 쓰기 데이터
dram_data_valid · rdata	입력	RD 후 RL(3) 엣지, RD 순서. data_valid로 진행 중 RD 수 카운트
q_empty	출력	두 큐 empty (HSL 레인 BIST가 읽기 전 대기)

Host 측 (valid/ready)

규칙	내용
수락	응답 tag 여유(진행 중 ≤ 15). 읽기는 읽기 큐 빈 슬롯(포워딩 시 불필요), 쓰기는 쓰기 큐 빈 슬롯 및 읽기 큐에 동일 주소 없음
동일 주소	쓰기 큐에 있는 주소의 읽기는 최신 쓰기 데이터로 즉시 응답(포워딩). 동일 주소 쓰기는 입력 순서 유지
완료 · 순서	읽기는 데이터 수신 시(포워딩은 수락 시), 쓰기는 수락 시(posted, rdata 0). 응답은 요청 순서
back-pressure	rsp_ready = 0이면 16-entry 버퍼에서 대기, 진행 중 15개면 수락 중지. rsp_err는 예약(0)

Spec ②: Timing Parameter 정의 및 담당 블록

단위: 컨트롤러 클럭 사이클 · 간격 = 두 이벤트 샘플 엣지 사이 사이클 수(모델 · 체커 동일 규약) · 자체 축소 값, JEDEC 아님

규칙	값	내용	담당 블록
tRCD	3	ACT → RD/WR, 동일 뱅크	mc_bank
tRP	3	PRE · PREA · auto-precharge → ACT · REF	mc_bank
tRAS · tRC	6 · 9	ACT → PRE · ACT → ACT, 동일 뱅크	mc_bank
tRTP · tWR	2 · 2	RD → PRE · WR → PRE ≥ WL + 1 + tWR = 4	mc_bank
tCCD_S / L	1 / 2	RD/WR → RD/WR, 다른 / 동일 그룹	mc_chan
tRRD_S / L	2 / 3	ACT → ACT (REFpb도 tRRD_S에서는 ACT)	mc_chan
tWTR_S / L	2 / 4	WR → RD, 다른 / 동일 그룹	mc_chan
tRTW · tFAW	4 · 8	RD → WR · 8사이클 창에 ACT 4개까지	mc_chan
tRFC · tRFCpb	12 · 8	REF 후 전 명령 · REFpb 후 해당 뱅크 ACT	mc_chan · mc_bank
tREFI · tREFIpb	64 · 16	뱅크별 tREFI당 1회 · per-bank는 16마다 REFpb	mc_refresh
RL · WL	3 · 1	RD → 읽기 데이터 · WR → 쓰기 데이터 (정확히)	mc_resp
RAA_IMT / MMT	4 / 12	RFM 시점 · 도달 시 ACT 금지	mc_bank · mc_sched

규칙 간 관계

- 데이터 버스 충돌 없음: RD 데이터는 RD + 3, WR 데이터는 WR + 1. tRTW 4로 WR 데이터를 앞 RD 데이터 이후에, tWTR로 RD를 앞 쓰기 데이터 이후에 배치
- WR → PRE 4 = WL 1 + 데이터 1 + tWR 2. write recovery는 데이터 이후부터 카운트
- auto-precharge: 장치가 PRE 허용 첫 엣지(tRAS, tRTP, write recovery)에 자체 close, 다음 ACT · REF는 해당 엣지 + tRP
- RFM 규모: tRC 9 기준 tREFI(64)당 뱅크별 ACT 최대 7개, RAA_IMT의 약 2배 (실제 장치와 동일 비율)
- refresh 비용이 실제보다 큼(tRFC / tREFI = 12 / 64). 절대값보다 버전 · 정책 간 상대 비교 용도

뱅크 그룹 (BG = 뱅크 번호의 MSB)

그룹	뱅크	동일 그룹 내 연속 명령
BG 0	b0 · b1	열 2사이클(tCCD_L) · ACT 3(tRRD_L) · WR → RD 4(tWTR_L)
BG 1	b2 · b3	연속 주소를 한 그룹에 모으는 매핑은 연속 읽기 50%로 제한(p.24)

Architecture: Request 처리 흐름 및 블록별 FF 구성

Block Diagram과 사이클별 결정 표는 p.22 · 본 페이지는 요청의 블록 통과 순서와 FF 1,616개 분포 (commit ab9d522)

요청 처리 흐름 (읽기: 1 → 7, 쓰기는 1에서 응답, 6에서 데이터)

단계	블록	동작
1 수락	mem_ctrl	주소 매핑 → 동일 주소 비교(match) → req_ready. 쓰기 · 포워딩된 읽기는 이 엣지에 응답 버퍼 저장
2 큐	mc_queue ×2	빈 슬롯에 push, age matrix 열 1개 기록, 동일 주소 쓰기 의존(q_dep)
3 다음 명령	mc_sched	hit → RD/WR, miss → ACT, conflict → PRE 및 ready 판정(mc_bank · mc_chan · RD 슬롯)
4 행 명령	행 버스	ACT · PRE는 요청을 큐에 유지한 채 발행. refresh · RFM 명령 우선
5 열 명령	열 버스	RD/WR 발행 엣지에 큐에서 pop (one-hot 유지)
6 데이터	mc_resp	RD tag를 read FIFO(4)에 저장, data_valid마다 1개 pop. WR 데이터는 WL 파이프
7 응답	mc_resp	16-entry 버퍼에 완료 표시, tag_retire 순서로 호스트에 반환

- **모듈**: mem_ctrl이 mc_queue ×2 · mc_xage · mc_wdrain · mc_sched · mc_bank ×4 · mc_chan · mc_refresh · mc_resp를 연결. 파라미터 13개(DEPTH, FR_FCFS, CMD_BUS, REF_MODE …)

FF 1,616개 구성 (v4, Yosys stat)

블록	FF	주요 레지스터
쓰기 큐 (mc_queue)	560	q_wdata 256 · q_wait 64 · q_row 64 · q_dep 64 · q_older 56 · q_col 32
응답 (mc_resp)	555	응답 데이터 512 · done 16 · read FIFO 23 · tag 4
읽기 큐 (mc_queue)	276	q_wait 64 · q_row 64 · q_older 56 · q_tag 32 · q_col 32
뱅크 ×4 (mc_bank)	104	open row 32 · 타이밍 카운터 48 · RAA 20 · open 4
채널 (mc_chan)	75	카운터 55 (그룹별 tCCD_L · tRRD_L · tWTR_L 30) · tFAW 창 20
쓰기 파이프 · refresh · 모드	46	쓰기 데이터 33 · refresh 11 · 모드 2

해석

- 데이터 저장 플롭(응답 512, 쓰기 데이터 256 + 32)이 전체의 50%. 응답 버퍼 단독 34%
- q_older의 대각선(자기보다 오래됨)은 상수 0이라 큐마다 64가 아닌 56
- WDRAIN = 1(기본)에서는 두 큐 간 age(mc_xage)와 bank idle 카운터 미사용으로 합성 시 제거

RTL ①: Request Queue · Age Matrix — 비교 Chain 없는 Oldest 선택

rtl/mc_queue.sv ×2 (읽기 8 · 쓰기 8) · mc_sched.sv의 oldest_in · 선택 결과는 one-hot으로 pop · tag · 데이터 선택까지 사용

rtl/mc_queue.sv · push

```
// q_older[j][i] = 1: j가 i보다 먼저 들어옴
if (push) begin
  q_bank[free_idx] <= push_bank;   // row, col, wdata, tag도
  for (int j = 0; j < DEPTH; j++) begin
    q_older[j][free_idx] <= q_valid[j];  // 있던 것은 모두 더 오래됨
    q_older[free_idx][j] <= 1'b0;
  end
end
// 쓰기 큐(ORDERED = 1): 같은 주소의 앞선 쓰기를 기다림
q_dep[free_idx][j] <= match[j];
match[j] = q_valid[j] && q_bank[j] == push_bank &&
           q_row[j] == push_row && q_col[j] == push_col;
```

rtl/mc_sched.sv · oldest_in (one-hot)

```
for (int i = 0; i < DEPTH; i++) begin
  r[i] = m[i];
  for (int j = 0; j < DEPTH; j++)
    if (j != i && m[j] && old[j][i]) r[i] = 1'b0;
end
// 집합 m 안에서 자기보다 오래된 원소가 없는 하나:
// 엔트리마다 AND-OR 한 단, 인덱스 인코딩 없음
```

Age Matrix 채택 배경

- tag 비교 chain으로 순서를 판정한 v2 초기 구현은 151 MHz. age matrix 적용 후 357 MHz (c1b8248)
- push 시 열 1개만 기록: 새 엔트리는 항상 최신. pop은 q_valid만 clear, 행렬의 이전 비트는 valid로 마스킹
- 대각선은 상수 0 → 합성 시 제거되어 큐마다 56 FF
- 두 큐 간 순서는 mc_xage(w_older_r). WDRAIN = 1(기본)에서는 한 번에 한 큐만 후보이므로 미사용

동일 주소 규칙 (기대값 판정 근거)

- 쓰기 → 쓰기: q_dep로 입력 순서대로 발행. 선행 쓰기가 남은 동안 RD/WR 후보에서 제외(hazard)
- 읽기, 쓰기 큐에 동일 주소: 최신 쓰기 데이터로 포워딩(p.11). 쓰기, 읽기 큐에 동일 주소: 해당 읽기 발행까지 포트에서 대기
- match는 이번 엣지 발행 엔트리도 비교: 수락 경로가 스케줄러 결정에 의존하지 않음(v3 타이밍 클로저, p.26)
- 회귀 1,200회: 포워딩 151,757 · 쓰기 후 동일 주소 쓰기 143,018 · 포트 대기 쓰기 470,830, 데이터 모두 일치

RTL ②: Scheduler — 요청별 다음 명령 · Ready 판정, 행 / 열 동시 발행

rtl/mc_sched.sv · FR-FCFS · 열 버스 · 행 버스 후보를 각각 선택해 동일 엣지에 발행 (버스별 우선순위 표는 p.22)

rtl/mc_sched.sv · 요청 (q, i)마다 (q = 0 읽기 큐, 1 쓰기 큐, b = 뱅크, g = 그룹)

```
hit  = q_valid && bank_open[b] && bank_row[b] == q_row;
conf = q_valid && bank_open[b] && bank_row[b] != q_row;
miss = q_valid && !bank_open[b];
col_rdy = live && hit && !hazard && !blocked[b] && bank_col_ok[b] &&
          !starve_bank[b] && (q ? chan_wr_ok[g]
                                : chan_rd_ok[g] && rd_slot_ok);
row_rdy = live && !blocked[b] &&
  (miss && bank_act_ok[b] && !bank_raa_mmt[b] && chan_act_ok[g] ||
   conf && bank_pre_ok[b] && chan_any_ok && !col_bank_rdy[b] &&
           (!guarded || starve_bank[b]));
ccand = |col_rdy ? col_rdy : xcol_rdy;  // 선택된 큐, 없으면 다른 큐
rcand = |row_rdy ? row_rdy : prep_rdy;
col_sel = oldest2(ccand);  row_sel = oldest2(rcand);
```

항목	의미
live · blocked	WDRAIN이 선택한 큐의 요청 · refresh 대상 뱅크
guarded	열린 row를 사용할 요청이 두 큐 중 존재(FR-FCFS)
col_bank_rdy	해당 뱅크에 열 명령 ready → PRE 미발행

보호 규칙

- row 보호: 열린 row를 사용할 요청이 남아 있으면 해당 뱅크 close 금지(FR-FCFS). FCFS는 더 오래된 hit만 보호
- 동일 뱅크 두 버스 충돌 방지: 열 명령 ready 뱅크에는 PRE 미발행
- starvation: 최고령 후보가 160사이클 초과 대기 시 oldest-first로 전환, 해당 뱅크의 후순위 hit 중지. 동일 시간 대기한 쓰기는 쓰기 모드로 전환
- refresh: 대상 뱅크는 ACT · RD/WR 차단, PRE → REFpb만 허용

page 정책 (PAGE_POLICY)

- open(기본): conflict · refresh로 close될 때까지 유지 · close: 동일 row 사용 요청이 없으면 RD/WR에 auto-precharge
- timer: 다른 명령이 없을 때, 16사이클 미사용이며 대기 요청도 없는 row를 PRE

- **회귀 1,200회**: 재정렬 3,386,363 · 행 · 열 명령 동일 엣지 발행 1,359,346 · 다른 그룹 연속 RD/WR 566,659

RTL ③: Bank · Channel Timing — 잔여 Cycle Counter 기반 제약 관리

rtl/mc_bank.sv ×4 · rtl/mc_chan.sv · 카운터 0 도달 다음 엣지에 즉시 발행 → 체커 측정 최소 간격 = Spec (p.15)

rtl/mc_bank.sv

```
// 카운터 = 명령이 합법이 될 때까지 남은 사이클, 0 = 다음 엣지에 합법
// 이벤트는 (spec - 1) 이상을 넣고, 제약이 겹치면 큰 값이 남는다
assign act_ok = !is_open && act_wait == '0;   // tRP · tRC · tRFCpb
assign col_ok =  is_open && col_wait == '0;   // tRCD
assign pre_ok =  is_open && pre_wait == '0;   // tRAS · tRTP · tWR
if (do_rd) pre_next = hold_for(pre_wait, tRTP);
if (do_wr) pre_next = hold_for(pre_wait, WL + 1 + tWR);
if (do_act) begin
  act_wait <= hold_for(act_wait, tRC);
  col_wait <= hold_for(col_wait, tRCD);
  pre_wait <= hold_for(pre_wait, tRAS);
end
if (do_pre)   act_wait <= hold_for(act_wait, tRP);
if (do_refpb) act_wait <= hold_for(act_wait, tRFCpb);
// auto-precharge: 장치가 닫는 엣지(pre_next + 1) + tRP까지 ACT 대기
```

- **카운터 폭**: 뱅크 4 bit(최장 대기 tRC − 1 = 8), 채널 5 bit(tRFC − 1 = 11). 채널의 짧은 카운터는 상위 비트 미토글로 mc_chan toggle 52%
- **FF**: 뱅크마다 26(open row 8, 카운터 12, RAA 5, open 1) × 4 = 104, 채널 75

rtl/mc_chan.sv · 그룹 g별 준비 신호

```
assign any_ok    = (rfc == '0);     // all-bank REF 뒤 tRFC
assign act_ok[g] = any_ok && rrd_s == '0 && rrd_l[g] == '0
                   && faw_free;
assign rd_ok[g]  = any_ok && ccd_s == '0 && ccd_l[g] == '0
                   && wtr_s == '0 && wtr_l[g] == '0;
assign wr_ok[g]  = any_ok && ccd_s == '0 && ccd_l[g] == '0
                   && rtw == '0;
// tFAW: 슬롯 4개. ACT마다 빈 슬롯에 FAW_LOAD, 하나라도 0이면 ACT 가능
if (do_act) faw_wait[faw_slot] <= FAW_LOAD;
```

체커와 측정 규약 통일 배경

- 체커도 '샘플 엣지 사이 사이클 수'로 측정. 카운터 만료 엣지에 즉시 발행하면 측정 최소 = Spec이 되어, 1사이클 지연 발행 비효율(v1의 handoff +1)도 수치로 확인
- 그룹별 _L 카운터와 채널 공통 _S 카운터를 분리해 동일 그룹 · 다른 그룹 규칙을 한 클럭에 동시 판정
- REFpb도 tRRD_S에서는 ACT로 카운트. 체커도 동일 규칙으로 측정

RTL ④: Per-bank Refresh · RFM — 대상 Bank만 차단, 타 Bank 동작 유지

rtl/mc_refresh.sv · mc_bank.sv(RAA) · mc_sched.sv(refresh · RFM 명령) · tREFIpb = tREFI / 4 = 16마다 REFpb 1회, 뱅크 순환(round-robin)

rtl/mc_refresh.sv

```
assign tick  = (refi == '0);              // 16사이클마다 1개 누적
assign quiet = (MODE == REF_BANK) ? !wanted[bank] : idle;
assign start = (owed != '0) && (quiet || owed >= 4'(URGENT));
assign hold  = busy_q || start;  // 그 뱅크: ACT · RD/WR 금지
refi <= tick ? REFI_LOAD : refi - 1'b1;
if (MODE == REF_BANK && do_ref) bank <= bank + 1'b1;
```

RFM (RowHammer 대응)

- RAA: 뱅크별 ACT 누적 수. ACT마다 +1, 해당 뱅크 refresh마다 −4(RAA_IMT, 최소 0)
- 12(RAA_MMT) 도달 뱅크는 ACT 금지. 열린 row 사용 요청이 없어지면 close 후 REFpb 발행
- RFM_EARLY(기본): RAA 4 이상이며 사용 요청이 없는 닫힌 뱅크에 선제 REFpb
- 비용: FF 20(뱅크당 RAA 5 bit), hotbank 처리량 약 5%(seed 3, 커밋 전 RFM 없는 v4 대비)

refresh 방식 (REF_MODE)

항목	per-bank (기본)	all-bank (v3)
주기	16사이클마다 REFpb 1개, 뱅크 순차	64사이클마다 REF 1개
발행 조건	대상 뱅크 대기 요청이 없을 때, 4개 누적 시 즉시	두 큐 empty 시, 4개 누적 시 즉시
순서	대상 뱅크만 PRE → REFpb	전체 뱅크 PREA → REF
이후	대상 뱅크만 tRFCpb 8, 타 뱅크 동작 유지	전 명령 tRFC 12 대기

측정

항목	값
뱅크당 refresh 간격 (directed)	평균 62.2, 최대 91 엣지 (tREFI 64)
밀린 refresh 최대	1 (v3 all-bank 4)
RAA 최대 (directed)	8 (RFM 시점 4, 한도 12)
RAA 한도 도달 (랜덤 1,200회)	131,879회, 한도 초과 ACT 0
REFpb · 동일 엣지 타 뱅크 RD/WR	1,674,822 · 111,515

RTL ⑤: Accept · Forwarding · Response — Posted Write, In-order 응답

rtl/mem_ctrl.sv(수락 · 포워딩) · mc_wdrain.sv(쓰기 묶음) · mc_resp.sv(tag · read FIFO · 쓰기 파이프 · 응답 버퍼)

rtl/mem_ctrl.sv · 수락과 포워딩

```
assign fwd       = !req_write && |q_match[1];  // 쓰기 큐에 같은 주소
assign collide   =  req_write && |q_match[0];  // 읽기 큐에 같은 주소
assign req_ready = tag_free && (fwd || q_has_free[req_write])
                   && !collide;
assign q_push    = {acc && req_write, acc && !req_write && !fwd};
// 포워딩 데이터: 같은 주소 쓰기 중 가장 나중 것 (age로 AND-OR)
fwd_sel[i] = q_match[1][i];
for (j ...) if (j != i && q_match[1][j] && q_older[1][i][j])
  fwd_sel[i] = 1'b0;
fwd_data |= {DATA_BITS{fwd_sel[i]}} & q_wdata[1][i];
```

쓰기 묶음 (mc_wdrain, wmode는 레지스터)

모드	조건
읽기 (기본)	읽기 우선
쓰기	쓰기 큐 6(WR_HI) 도달 시 2(WR_LO)까지, 읽기 없음, 160사이클 대기 쓰기 존재 시
(공통)	차단이 아닌 우선순위: 선택된 큐가 사용하지 못하는 슬롯은 다른 큐가 사용

응답부 (mc_resp)

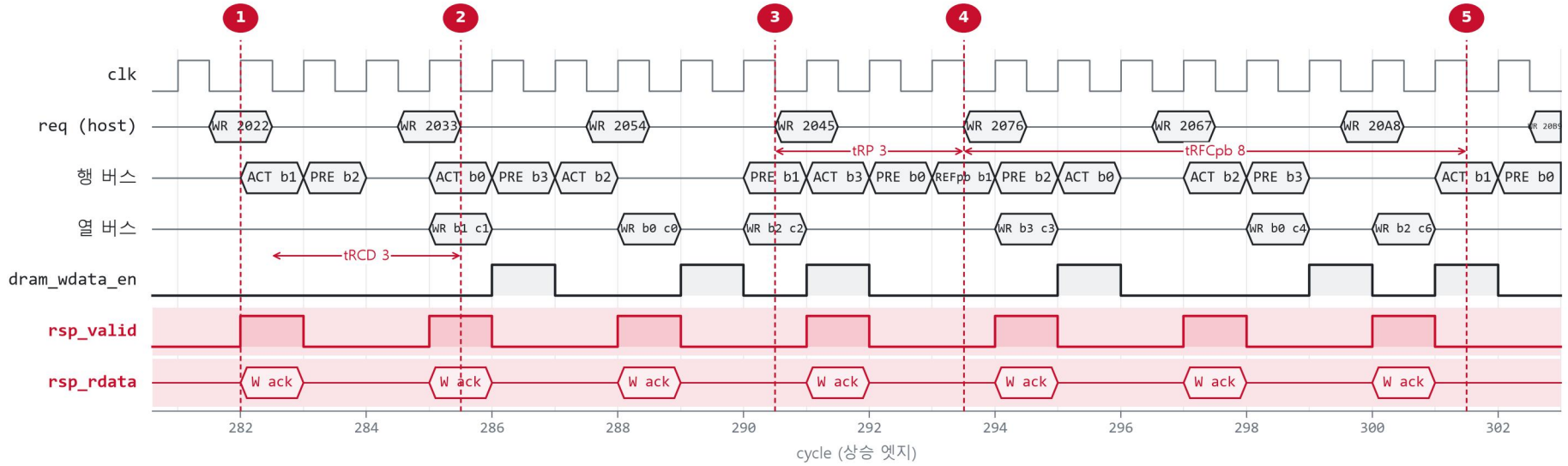
- tag 4 bit, 진행 중 최대 15개(tag_free = tag_alloc + 1 ≠ tag_retire). v1 tag wrap-around 수정 규칙 유지(p.17)
- RD는 tag를 read FIFO(깊이 RD_PIPE = 4)에 저장, data_valid마다 1개 pop. PHY 데이터 지연이 길어도 순서만 맞으면 동작(HSL, p.20)
- WR 데이터는 WL 단 파이프에서 wdata_en과 함께 출력
- 완료 응답은 16 × 32b 버퍼에서 tag_retire 순서로 반환. 버퍼가 FF의 34%

회귀 발생 횟수 (1,200회, 기능 bin 합계)

상황	합계
쓰기 큐에서 포워딩된 읽기	151,757
읽기 큐 동일 주소로 포트에서 대기한 쓰기	470,830
tag 한도(15)에서 요청 대기 (사이클)	13,517,082
WR 4개 이상 연속 (쓰기 묶음)	319,245

파형 ①: Write 흐름 — 행 · 열 버스 동시 발행 및 Per-bank Refresh

make sim WAVES=1의 Verilator VCD(RTL 44a0bb3, directed 로그 ab9d522와 동일) · 시나리오 5 직전 쓰기 20개 중 cycle 281–302 · XOR 매핑



bank 1 refresh (엣지)

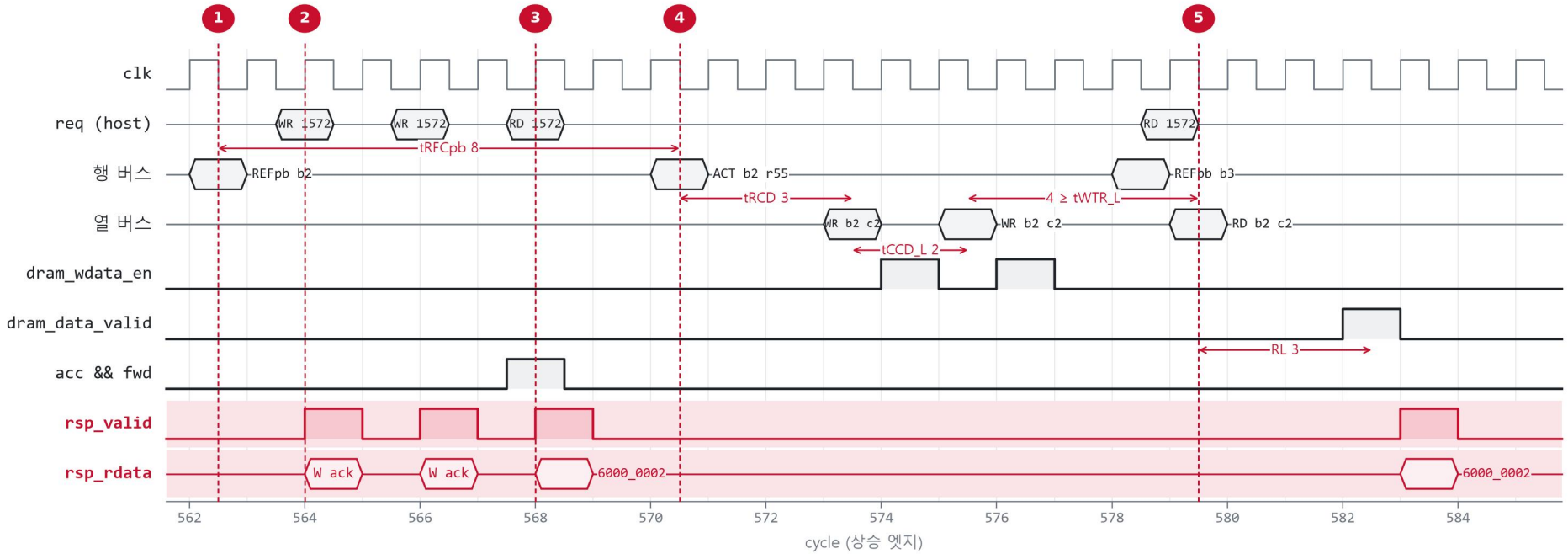
명령	엣지
ACT b1 r80	283
WR b1 c1	286
PRE b1	291
REFpb b1	294
ACT b1 r81	302

- **REFpb는 16사이클마다 다음 뱅크**: b0 278 → b1 294

- ① 엣지 282에 WR 2022 수락, 283에 W ack (posted). 데이터는 이후 WR 발행 시 출력
- ② 엣지 286: 행 버스 ACT b0와 열 버스 WR b1 c1 동일 엣지 발행. 버스가 하나면(v3) 1사이클씩 순차 발행
- ③ 엣지 291: bank 1 refresh 순번. PRE b1(WR 286 + 4, ACT 283 + tRAS 6 이후)과 WR b2 c2 동일 엣지
- ④ 엣지 294: REFpb b1 = PRE + tRP 3. 그동안 b0 · b2 · b3는 ACT · PRE · WR 정상 처리
- ⑤ 엣지 302: ACT b1 r81 = REFpb + tRFCpb 8. refresh 차단 대상은 bank 1뿐 (all-bank면 전체 row close 후 약 25사이클 정지)
- **간격**: ACT b1 → WR b1 = tRCD 3, WR 다음 엣지에 wdata_en (WL 1). 육각형은 해당 명령이 버스에 있는 사이클, 샘플은 오른쪽 상승 엣지

파형 ②: Refresh 직후 동일 주소 — Posted Write · 포워딩 · Bank Timing

시나리오 6, cycle 562–585 (RTL 44a0bb3) · 1572 = bank 2 · row 55 · col 2 · 해당 뱅크 REFpb 직후 시작으로 ACT 시점 고정



간격 (규칙)

구간	사이클
REFpb → ACT	8 tRFCpb
ACT → WR	3 tRCD
WR → WR	2 tCCD_L
WR → 데이터	1 WL
WR → RD	4 ≥ tWTR_L
RD → 데이터	3 RL

- **acc && fwd**: 수락 엣지에 포워딩된 읽기

- ① 563 REFpb b2: bank 2는 tRFCpb 8 동안 ACT 금지
- ② WR 1572 2건을 564 · 566에 수락, 다음 엣지에 W ack. 두 쓰기는 쓰기 큐에 대기 중
- ③ 568 RD 1572: 쓰기 큐에 동일 주소 → 최신 쓰기 데이터 6000_0002로 569에 응답, 열 버스 RD 없음
- ④ 571 ACT b2 r55(563 + 8) → 574 WR(tRCD 3) → 576 WR(동일 뱅크 tCCD_L 2). 동일 주소 두 쓰기는 입력 순서 유지(q_dep)
- ⑤ 579 재읽기는 쓰기 발행 이후라 580에 RD b2 c2(576 WR 후 4클럭으로 tWTR_L 충족), 583 data_valid(RL 3), 584 응답: hit RD 5사이클

검증 ①: Directed 시나리오 · Pin-level Checker · 응답 판정 기준

모든 시뮬레이션에 bind 체커 2종 적용 · directed 6개(make sim)와 랜덤 TB(make regress, p.23)가 동일 체커 사용 · commit ab9d522

directed 시나리오 (tb/tb_mem_ctrl.sv)

#	시나리오	확인 내용
1	page hit WR → RD	쓰기 DRAM 반영 후 read-back
2	page miss (bank 0 row 변경)	PRE → ACT 경로, 새 row와 이전 row의 데이터
3	multi-bank + conflict	bank 0 row 변경 후에도 bank 1 데이터 유지
4	4-bank write burst	ACT 발행 후 WR 대기 중인 뱅크 동시 2개 이상
5	응답 back-pressure	rsp_ready 150사이클 0, read 20건(tag 16 초과): 20/20 순서 · 데이터
6	포워딩	동일 주소 쓰기 2건 후 읽기: 최신 쓰기 데이터, DRAM RD 0건

```
make sim 출력 (발췌, Verilator 5.052)
[TB] peak banks opening at once=2
[TB] back-pressure OK (20/20 in order, 15 REF during the test)
[TB] forwarded RD OK (0x60000002)
[TB] access_err count=0 OK
[TCHK] PASS: 0 violation(s), 0 waived
[HCHK] PASS: controller violations 0, host violations 0
[TB] PASS — hit/miss/multi-bank/burst/back-pressure/forwarding OK
```

체커 구현 세부 (p.23 요약 외)

- 한 엣지의 두 버스 명령은 열 명령부터 판정: 동일 뱅크 PRE + RD는 tRTP 위반, ACT + RD는 '닫힌 뱅크에 RD'로 검출
- 밀린 refresh는 뱅크별 '경과 tREFI 주기 수 − 수행 refresh 수'. 한도 8은 DDR4 REF postpone 개수 준용
- RAA도 체커가 독립 카운트(ACT +1, 해당 뱅크 refresh −4). 12 도달 뱅크에 ACT 발행 시 위반
- --assert 없이 빌드 시 assertion 비활성 상태로 통과 가능 → 시작 시 활성 여부 확인, 비활성이면 FAIL

응답 판정 (랜덤 TB의 scoreboard)

- posted write: 응답은 즉시 반환되나 해당 WR의 핀 발행 여부를 별도 확인. 마지막 응답 후 2,000사이클 내 미발행 쓰기는 FAIL
- 포워딩: 수락 엣지의 핀 활동까지 기록한 하강 엣지에서 예측. 쓰기 큐에 해당 쓰기가 있는데 RD 발행 시 FAIL, 없는데 RD 없이 응답 시 FAIL
- 열 버스에는 row 필드 없음: 다른 row로 간 RD/WR는 TB 매칭(열린 row 기준 주소 계산)으로 검출

검증 ②: Timing 18종 · Protocol 10종 검사 — 측정 최소 간격 = Spec

dram_timing_checker(bind) · make sim 로그(commit ab9d522) · 측정 최소 = Spec: 카운터 만료 엣지에 즉시 발행 · 랜덤 1,200회 위반 0

규칙	Spec	측정 최소	검사 간격 수
tRCD ACT → RD/WR	3	3	61
tRP PRE · PREA · AP → ACT · REF	3	3	53
tRAS ACT → PRE · PREA	6	6	53
tRC ACT → ACT, 동일 뱅크	9	9	50
tRTP RD → PRE	2	2	29
tWR WR → PRE (WL + 1 + tWR)	4	4	28
tRRD_S ACT · REFpb → ACT · REFpb	2	2	88
tRRD_L ACT → ACT, 동일 그룹	3	3	50
tFAW ACT 4개 창	8	10	50
tCCD_S RD/WR → RD/WR	1	2	60
tCCD_L 동일 그룹	2	2	59
tWTR_S WR → RD	2	4	31
tWTR_L WR → RD, 동일 그룹	4	4	31
tRTW RD → WR	4	4	29
tRFCpb REFpb → 해당 뱅크 ACT · REFpb	8	8	34
tRFC REF → 다음 명령	12	–	0
RL · WL 데이터 지연 (정확히 일치)	3 · 1	3 · 1	31 · 30

프로토콜 10종 (위반 0)

- 열린 뱅크에 ACT · 닫힌 뱅크에 RD/WR · 열린 row가 있는 REF · 열린 뱅크에 REFpb
- RD 없는 읽기 데이터 · 누락된 읽기 데이터 · WR 없는 쓰기 데이터 · 누락된 쓰기 데이터
- 뱅크당 밀린 refresh > 8 · RAA 12 이상인 뱅크에 ACT

make sim 종료 시 버스 집계

```
[TCHK] row bus: ACT 54, PRE 53, PREA 0, REF 0, REFpb 37;
       column bus: RD 31, WR 30; both on one edge 27
[TCHK] refresh interval per bank avg 62.2, max 91 edges
[TCHK] peak RAA 8 (RFM due at 4, limit 12)
```

- **측정 최소 > Spec**(tFAW 10, tCCD_S 2, tWTR_S 4)은 directed 시퀀스에 더 근접한 간격이 없었음을 의미. 랜덤 회귀에서 더 많은 간격 검사
- **tRFC 0회**: 기본 빌드는 per-bank로 REF 미사용. refab · bus1_refab 회귀에서 검사
- **waiver 0**: v1은 refresh 2개 항목을 known issue로 별도 집계, v2에서 수정 후 해제

검증 ③: Functional Coverage 32 bin 및 Coverage Hole 분석

랜덤 TB mc_coverage의 bin 합계 · 회귀 1,200회 · Code Coverage 표는 p.23 · Hole은 tools/cov_summary.py --holes로 추출 (commit ab9d522)

bin	합계	Hit 실행 수
read hit · miss · conflict	894,831 · 734,816 · 1,213,055	1200 · 1200 · 961
write hit · miss · conflict	1,102,895 · 689,846 · 1,212,800	1200 · 1200 · 966
15개 in flight (tag 한도)	786,858	1192
재정렬 (후순위 요청 RD/WR 선발행)	3,386,363	1200
DRAM 버스에 RD 2개 · 3개 동시	942,730 · 50,449	1140 · 793
행 · 열 명령 동일 엣지	1,359,346	1080
다른 그룹 연속 RD/WR	566,659	1185
REFpb · 동일 엣지 타 뱅크 RD/WR	1,674,822 · 111,515	1160 · 1053
RFM REFpb · RAA 12 도달	709,148 · 131,879	979 · 800
auto-precharge RD/WR (close)	211,907	60
REF 앞 PREA (all-bank 빌드)	40,059	120
응답 대기 중 · rsp_ready 0 상태 refresh	1,703,953 · 663,627	1200 · 1200

- **Hit 실행 수가 적은 bin**은 해당 기능이 없는 빌드 때문: 두 버스 bin 미발생 120회는 bus1 계열, PREA는 all-bank 2개 빌드만. conflict 없는 실행은 대부분 rowhit 패턴

모듈별 Code Coverage (p.23 표 외 2개 모듈, directed → + 랜덤)

모듈	Line	Branch	Toggle
mc_chan (그룹 타이밍)	100 → 100%	96.2 → 100%	46.3 → 52.1%
mc_refresh	100 → 100%	83.3 → 91.7%	89.1 → 94.0%

- **mc_chan toggle 52%**: 모든 채널 카운터가 tRFC − 1 = 11 기준 5 bit라 짧은 카운터의 상위 비트 미토글. 기능 Hole 이 아닌 비트 폭 문제
- **mc_refresh branch 91.7%**: 밀린 refresh 카운터 포화(15). REF_URGENT 4부터 발행하므로 설계상 미도달
- **나머지 Hole**(older() 분기, WL > 1 쓰기 단, dram_model 오류 경로)은 p.23 하단 참조

해소된 Hole (v1 → v4)

- out-of-order dispatch와 write-hit: 재정렬 3,386,363 · write hit 1,102,895, 응답 순서 유지
- 지연 REF · PREA 없는 REF(v1): v2부터 tREFI마다 발행 및 선 close, 둘 다 위반으로 집계 → 0
- starvation · FCFS 경로: starve24 · fcfs · fcfs_nodrain 빌드로 회귀 · 커버리지 수행

Debug ①: 응답 Back-pressure 시 Tag Wrap-around 이슈 (v1)

coverage.md의 gap 항목(back-pressure)을 실험 TB로 재현해 발견 · 수정 fa08330, 회귀 7e2d4b5 · 동일 15개 규칙이 v4의 tag_free

증상 → 원인 → 수정

- **증상**: rsp_ready 150사이클 deassert 상태에서 read 20건 → 해제 후 앞 4건 데이터 오류, 뒤 4건 응답 누락(16/20)
- **원인**: req_ready가 큐 빈 슬롯만 판단. dispatch로 큐가 계속 비워져 응답이 막혀도 요청을 수락했고, 4-bit tag_alloc이 16을 순환해 반환 전 tag 재할당 → 17번째 read가 첫 read의 버퍼 엔트리 overwrite
- **수정 (fa08330)**: 처리 중 요청 15개 도달 시 수락 중지
- **회귀 (7e2d4b5)**: 시나리오 5로 고정. 수정 전 RTL은 첫 응답에 0x50000010(17번째 read 데이터) 반환으로 FAIL, 수정 후 20/20 순서 일치



- **파형 (v1)**: rsp_ready 해제 구간. 위 3줄은 수정 전, 아래 2줄은 수정 후. 수정 전에는 tag 4 위치에 17번째 read 데이터(10)가 출력되고 16건 후 rsp_valid deassert. 수정 후에는 00부터 13까지 20건 순서대로 출력

v1 mem_ctrl.sv (diff)

```
- req_ready = free_found && !ref_due && !ref_active;
+ req_ready = free_found && !ref_due && !ref_active &&
+              (TAG_BITS'(tag_alloc + 1'b1) != tag_retire);
```

- **v4**: mc_resp의 tag_free가 동일 규칙. 랜덤 회귀의 'tag 한도 요청 대기' bin(1,192회 실행)이 이 상황을 반복 재현, HSL SMOKE도 칩에서 '요청 15개 시 ready 0' 확인

Lessons Learned

- Flow control은 실제 자원(응답 버퍼) 기준으로 판단 필요. 큐의 빈 슬롯이 응답 버퍼 여유를 의미하지 않음
- gap 목록의 상황을 재현 TB로 직접 구성한 것이 발견 계기, 해당 TB를 회귀 시나리오로 편입

Debug ②: Issue Tracking — 증상 · 원인 · 조치 (v1 → v4, HSL 통합)

commit 이력 기반 버그 · 규칙 · 성능 · 타이밍 · 검증 · 통합 이슈 정리 · v1 재검증 항목(B–E)의 해소 시점 병기

분류	증상	원인	조치 (commit)
v1 버그	back-pressure 시 응답 데이터 오류 · 누락	tag wrap-around: 큐 빈 슬롯만으로 수락 판단	진행 중 15개 한도 (fa08330, p.17)
v1 규칙	REF가 tREFI 초과 지연(누적 최대 549), PREA 없이 REF	큐 empty 시에만 REF (drain-then-REF)	tREFI마다 refresh 요구 누적 · 발행 + PREA (v2 893c9a4), 체커 위반 집계
v1 성능 (B)	closed RD 10사이클(v0 7)	큐 단계 +1, wait → issue handoff +1, 응답 버퍼 +1	명령 단위 스케줄러 (v2): 8사이클, 연속 읽기 3.4×(RD 4개)
v1 검증 (C)	뱅크 병렬성 검사가 v0에서도 PASS	'열린 뱅크 ≥ 2'가 burst 전에 이미 참	동시 동작 뱅크 수로 판정 (7e2d4b5)
v1 검증 (D)	out-of-order · write-hit 경로 미도달	blocking task라 큐 점유가 최대 1	non-blocking 랜덤 TB: 재정렬 3,386,363 · write hit 1,102,895
v2 타이밍	age 비교 경로 151 MHz	tag 비교 체인	age matrix, 357 MHz (c1b8248, p.7)
v3 타이밍	v3 초기 구현 208 MHz	쓰기 모드 → 후보 → pop → 동일 주소 비교 → req_ready, fanout 58 · 104	수락 경로 분리 · 후보 통합 · one-hot 유지 → 337 MHz (1b0943a)
v3 성능	쓰기 묶음으로 처리량 최대 12% 저하	반대 큐를 완전 차단	우선순위 방식, 빈 슬롯은 반대 큐가 사용
v3 TB	포워딩 판정 1사이클 불일치	엣지 카운터를 class 핸들로 읽음	엣지 번호를 인자로 전달, RTL 변경 없음 (상세 p.23)
통합	ModelSim 10.1d 컴파일 실패	폭이 파라미터인 캐스트 21곳	타입 있는 localparam (44a0bb3)
통합	HSL 레인 BIST 링크 미경유 가능	쓰기 큐 주소의 읽기는 큐에서 응답	q_empty 출력 (44a0bb3)
Open (E)	dram_model 오류 경로 미검증	컨트롤러가 해당 명령을 발행하지 않음	모델 직접 구동 negative test

Design Trade-off: 설계 선택별 이점과 비용

수치는 기본 빌드 회귀 · 포화 측정(commit ab9d522) · 기능별 제거 빌드의 처리량 · 면적 비교는 p.26

선택	이점	비용
명령 단위 스케줄러	요청별 다음 명령 즉시 발행. handoff 없이 동일 row hit 연속 처리	매 사이클 16개 엔트리를 평가하는 조합 논리
행 · 열 명령 버스	PRE · ACT와 RD/WR 간 슬롯 경합 없음. per-bank refresh와 함께 row hit 포화 +27%	명령 핀 2세트, 동일 뱅크 PRE · 열 명령 충돌 방지 논리
per-bank refresh	refresh 중 타 뱅크 동작 유지. 밀린 refresh 최대 1개	refresh 명령 4배, 뱅크별 순차 close
RFM (RAA 카운터)	RowHammer 한도 준수. 한도 도달 131,879회에도 위반 0	FF 20개, hotbank 처리량 약 5% (seed 3)
FR-FCFS와 row 보호	row hit 우선 처리로 hit rate · 처리량 향상	오래된 conflict가 밀릴 수 있어 starvation 보호가 필요
읽기 · 쓰기 큐 분리	쓰기가 읽기 슬롯을 점유하지 않음. 읽기 큐는 데이터 미저장	엔트리 16개, 두 큐 사이 순서(mc_xage, WDRAIN = 0일 때만 합성)
posted write + 포워딩	쓰기 응답 1사이클, RAW 읽기 37,434건이 DRAM 없이 응답(기본 빌드 회귀)	동일 주소 읽기가 큐에 있으면 쓰기를 포트에서 보류
쓰기 묶음	방향 전환 −19%, 읽기 latency −1.5% (회귀 6패턴)	처리량 이득 없음 (FR-FCFS가 이미 묶어 처리)
XOR 주소 매핑	v2 매핑보다 stride 3×, 연속 주소 +40% (v4 회귀)	연속 16 word가 한 뱅크 그룹 → 연속 읽기가 tCCD_L로 제한
one-hot 선택	index 인코딩 · 디코딩 단계 제거 (208 → 337 MHz 개선의 일부)	데이터 선택이 AND-OR 트리
RD 4개 in flight	연속 읽기 3.4× (v1 대비)	read-return FIFO, PHY 왕복이 길면 부족 (p.20)
16 × 32b 응답 버퍼	Host 인터페이스 단순 (요청 순서)	FF의 34%

HSL 통합: 동일 RTL의 Base Die 적용 — DFI 형태 경계 및 도출 과제

High-Speed Link Lite가 본 RTL을 동일 파일로 사용(로직 44a0bb3, 주석 422036a) · HSL 수치는 HSL e61e90d, Nangate45, Pre-layout

Interface: 컨트롤러 신호의 HSL 연결 대상

컨트롤러	HSL
dram_rcmd · rbank · row	CA 레지스터 → pin_rca {cmd, bank, row}, 1클럭 후
dram_ccmd · cbank · col · ap	pin_cca {cmd, bank, col, ap}
dram_wdata_en · wdata (WL 1)	DBI → 레인 shift → 동기 2:1 TX 기어 → 1.6 GHz beat
dram_data_valid · rdata	RDQS 캡처 → Gray FIFO → ECC 디코더 → rddata_valid. 지연 비고정, 순서 보장으로 동작
q_empty	레인 BIST가 쓰기 16개 후 큐 empty까지 대기 후 읽기
req_* · rsp_*	MISSION · phy_en일 때만 열리는 요청 포트. 시험 엔진 동작 중에는 엔진이 점유

통합 시 본 저장소 수정 사항 (44a0bb3)

- ModelSim-Altera 10.1d가 폭이 파라미터인 캐스트(BANK_BITS'(b) 등) 21곳 파싱 실패 → 타입 있는 localparam 적용, 회귀 재수행 후 반영
- q_empty 출력: 쓰기 큐 잔류 주소의 읽기는 큐에서 응답해 링크를 경유하지 않기 때문

HSL 통합 도출 과제

- 800 MHz 목표에서 최소 주기 2.18 ns(459 MHz). 한 클럭 내 명령 선택 · 전 뱅크 타이머 갱신 루프(약 35단) → DFI 1 : 2(400 MHz, 클럭당 명령 슬롯 2개)
- 읽기 대역폭: RD 1개가 발행 클럭부터 data_valid까지 슬롯 점유. 본 저장소 모델은 3클럭, HSL PHY는 8클럭(CA 레지스터 1 + 핀 RD → rddata_valid 7). RD_PIPE 4로 읽기 32개에 97클럭 → RD_PIPE를 PHY 왕복에 맞춤
- Host 입력 → req_ready 조합 경로(포워딩 비교, 읽기 큐 충돌) −0.521 ns → Host 경계 레지스터 슬라이스

동일 RTL, 2개 합성 Flow

Flow	최소 주기	매핑
본 저장소 make ppa	2.874 ns (348 MHz)	mem_ctrl만 flatten, 타이밍 목표 · 팬아웃 버퍼 없음
HSL 칩 STA	2.18 ns (459 MHz)	칩 전체 계층 유지, abc -D 1250, 팬아웃 버퍼 트리

- **둘 다 Nangate45 typical, Pre-layout**. 차이 요인별 기여도는 분리 측정하지 않음

Version별 Latency: 구조 변경에 따른 경로별 Cycle 변화

Host 수락 ~ 응답 handshake(rsp_ready 1, directed TB 단일 요청) · v0 64e73e0, v1 2eabd33, v2 c1b8248, v3 1b0943a, v4 ab9d522

경로 (사이클)	v0	v1	v2	v3	v4	변경 사유
page hit RD	4	6	5	5	5	v1 큐 · 뱅크 엔진 단계, v2 명령 단위 스케줄러
closed-bank RD	7	10	8	8	8	v1 +3 (아래 분해)
row-conflict RD	11	14	11	11	11	
WR (뱅크 상태 무관)	6–10	9–14	6–9	1	1	v3부터 posted write
쓰기 큐 주소의 RD	–	–	–	1	1	v3부터 포워딩
refresh 간섭 읽기				23	5	v4는 refresh가 대상 뱅크만 차단

closed-bank RD 분해 (v0 → v1, 사이클)

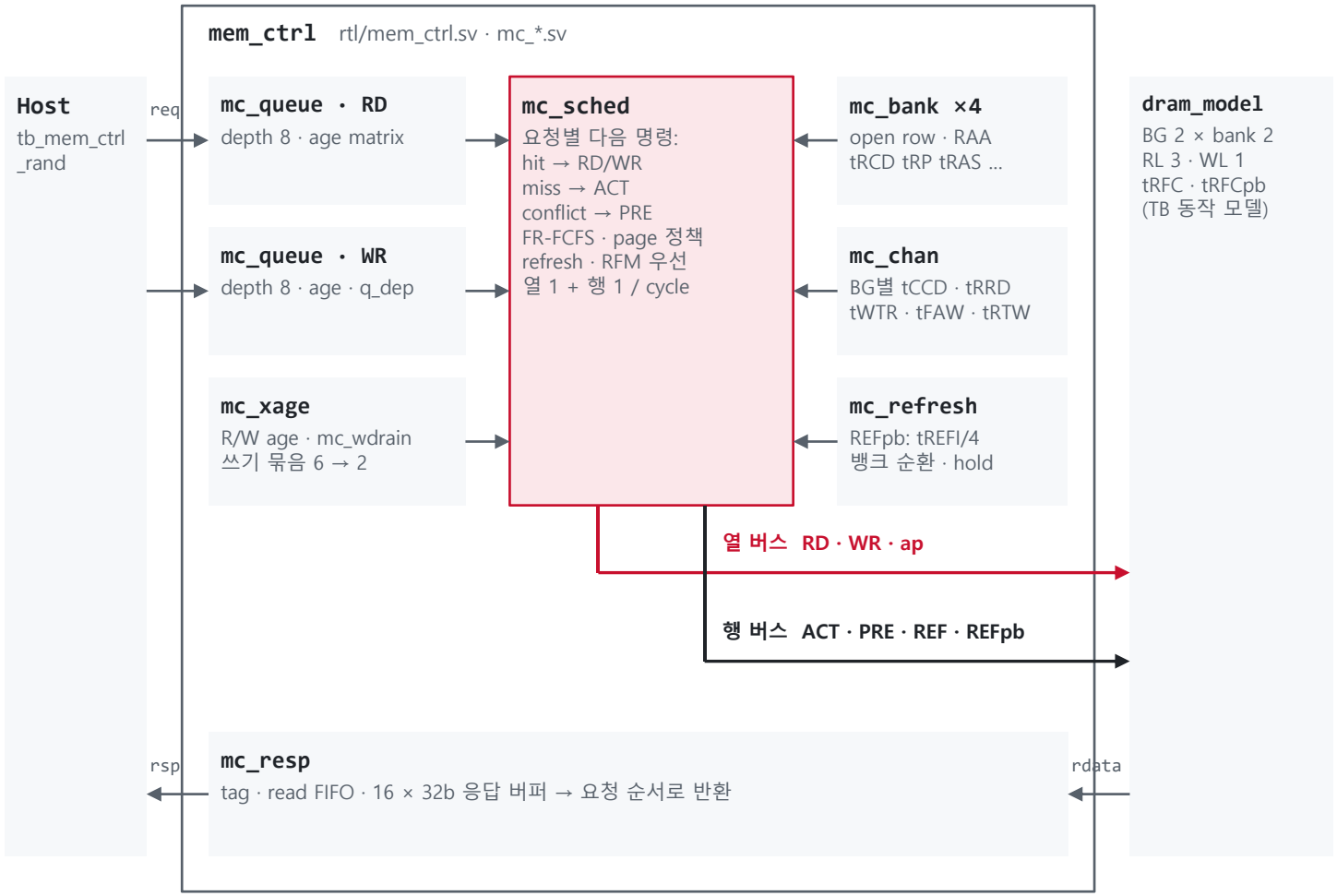
구간	v0	v1	v1 증가 사유
수락 → ACT	1	2	큐 단계 +1
ACT → RD	3	4	wait → issue handoff +1
RD → 응답	3	4	응답 버퍼 +1

해석

- v0 단일 FSM은 단일 요청 처리로 latency는 최단이나 병렬 처리 불가: 4-bank burst 35 클럭, v1은 17클럭(2.06×)
- v1은 뱅크 병렬성 확보 대신 경로별 2–3사이클 증가. v2에서 handoff 제거, RD 4개 in-flight 지원
- v3 · v4는 경로 latency를 유지하며 쓰기 · RAW 읽기 · refresh 영향 감소. 처리량 비교는 p.24

v4 Architecture: 요청 큐 2개 → 명령 단위 Scheduler → 행 · 열 명령 버스

요청은 RD/WR 발행 시까지 큐에 유지, 스케줄러가 매 사이클 요청별 다음 명령을 계산해 열 버스 · 행 버스에 각 1개 발행



사이클별 결정 (요청 단위)

요청 상태	다음 명령	Ready 조건
hit **row 열림**	RD / WR	tRCD, 그룹별 tCCD · tWTR(RD) · tRTW(WR), 동일 주소의 선행 쓰기 없음
miss **뱅크 닫힘**	ACT	tRP · tRC · tRFCpb, 그룹별 tRRD, tFAW, RAA < 12
conflict **다른 row**	PRE	tRAS · tRTP · tWR, 해당 row 사용 요청이 두 큐에 없음

- **열 버스**: 쓰기 묶음 모드가 선택한 큐의 RD/WR, 없으면 다른 큐의 RD/WR 중 최고령 요청
- **행 버스**: refresh(PRE → REFpb) > RFM > 선택된 큐의 ACT/PRE > 다른 큐의 ACT · 미사용 row PRE > timer PRE
- **최고령 판정**은 age matrix의 AND-OR 1단. 선택 결과는 one-hot 그대로 큐 pop · tag · 데이터 선택까지 사용
- **posted write · 포워딩**: 쓰기는 수락 엣지에 응답, 쓰기 큐에 동일 주소가 있는 읽기는 DRAM 접근 없이 응답

검증 Infra: 독립 Bind Checker + Random TB, 회귀 1,200회 위반 0건

체커는 RTL · TB 수정 없이 DRAM 핀만 관측해 bind · 동일 TB로 기본 빌드와 파라미터 변형 15종 모두 수행 (commit ab9d522)

DRAM 타이밍 · 프로토콜 체커 (bind)

두 명령 버스만 관측해 open row · 명령 시각 · 뱅크별 refresh · RAA를 독립 재구성, **타이밍 18종**(뱅크 그룹 S/L, tRFCpb 포함)과 **프로토콜 10종**(RAA 한도 포함) 검사. auto-precharge는 장치의 precharge 엣지를 계산해 다음 ACT의 tRP 측정. mem_ctrl 내부 신호는 참조하지 않음.

랜덤 TB (UVM 구조 기반 class TB)

sequence → driver → DUT → monitor → scoreboard(참조 메모리). 주소 매핑을 RTL과 독립 구현, 열 버스에 없는 row는 행 버스의 ACT로 추적. posted write는 WR 실제 발행 여부, 포워딩은 RD 미발행 여부를 핀에서 확인. **기능 bin 32개**, seed 재현.

Host SVA · directed 시나리오

valid/ready concurrent assertion 4개 + cover 5개. directed 6개: hit, miss, conflict, 4-bank burst, 응답 back-pressure(tag 16개 초과), 포워딩.

디버그: 포워딩 판정 78회 중 77회 FAIL (v3)

bind probe로 cycle 265 확인. 쓰기 WR 발행 엣지에 동일 주소 읽기가 수락되는 경우, 핀 모니터가 같은 엣지에 갱신되는 카운터를 class 핸들(vif.cyc)로 읽어 판정이 불안정해진 **TB race** → TB top이 엣지 번호를 인자로 전달. RTL 변경 없이 960/960 PASS.

랜덤 회귀 (make regress)

빌드	실행	PASS	위반
기본: 두 버스 · per-bank · RFM · 쓰기 묶음 · open · XOR	300	300/300	0
refresh · 버스: bus1 · refab · bus1_refab · rfm_late	240	240/240	0
스케줄링: fcfs · fcfs_nodrain · nodrain · starve24	240	240/240	0
구조: rdpipe1 · depth4	120	120/120	0
page · 매핑: close · timer · map_brc · map_rbc · map_rcb	300	300/300	0
합계 16 빌드 · 600만 요청	1,200	1,200/1,200	0

재정렬 3,386,363 · 포워딩 151,757 · 행 / 열 동시 발행 1,359,346 · RAA 한도 도달 131,879회. 응답 순서와 데이터 모두 일치.

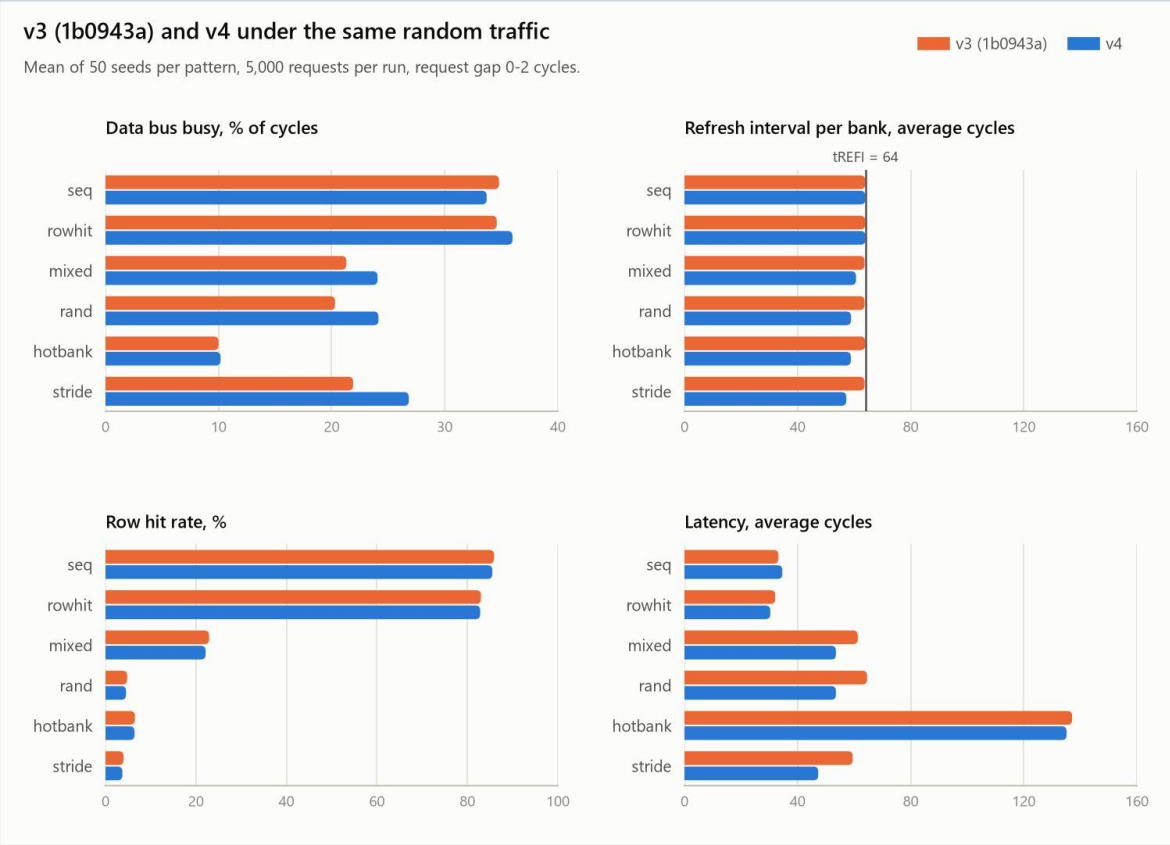
Code Coverage (make cov, directed → + 랜덤)

범위	Line	Branch	Toggle
컨트롤러 (mem_ctrl + mc_*)	94.8 → **98.7%**	81.8 → **98.2%**	48.5 → **93.1%**
mc_sched	91.9 → **100%**	62.5 → **96.9%**	43.2 → **91.5%**
RTL 전체 (dram_model 포함)	88.9 → **96.0%**	77.7 → **94.1%**	53.9 → **93.7%**

잔여 Hole: older() 분기 2개(루프 전개 시 상수로 축약), 밀린 refresh 카운터 포화(설계상 미도달), WL > 1 쓰기 파이프 단, dram_model 오류 경로(negative test 필요).

성능 분석: 동일 Traffic 기준 v3 → v4 비교, 포화 Throughput 기능별 분해

랜덤 회귀 평균(seed 50 × 5,000건)과 포화(요청 간격 0, seed 1) · 표 중앙 4열은 v4 파라미터 변형 빌드



v3 = commit 1b0943a(뱅크 그룹 없는 장치), v4 = ab9d522. conflict 비중이 큰 트래픽의 개선 폭이 최대(rand +19%, stride +22%). seq만 −3%: 동일 그룹 내 연속 RD/WR의 tCCD_L(2) 제약. 뱅크당 refresh 간격 57–64(RFM의 REFpb 포함), 밀린 refresh 최대 1개.

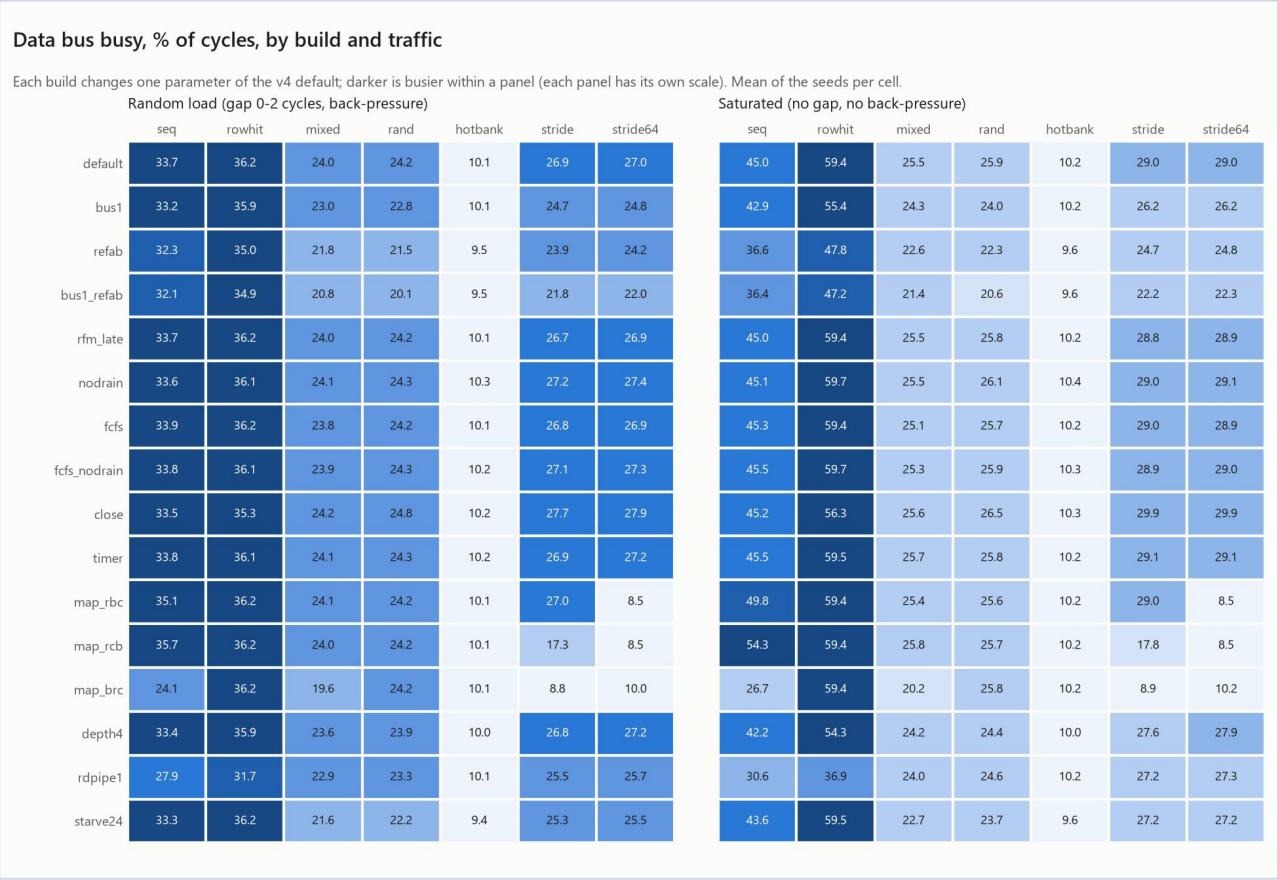
포화 처리량: 2개 기능 단계별 적용 (데이터 버스 사용률)

트래픽	v3 방식 bus1_refab	버스 분리만 refab	per-bank만 bus1	v4	v4 + row:col:bank
row hit, 읽기만	63.3%	65.6%	71.8%	**80.7%**	80.7%
row hit, 쓰기만	60.4%	62.1%	72.3%	**80.7%**	80.7%
row hit, 반반	47.3%	47.8%	55.2%	**59.6%**	59.6%
연속 주소, 읽기만	39.7%	40.2%	48.0%	**50.0%**	**78.5%**
랜덤 주소, 읽기만	21.9%	23.4%	24.7%	**26.5%**	27.1%
패턴 혼합, 반반	21.7%	22.7%	24.3%	**25.7%**	25.6%

- **per-bank refresh 효과 최대**: row hit 읽기 63.3 → 71.8%. all-bank는 tREFI마다 전체 row close 후 약 25사이클 정지
- **두 기능의 시너지**: 버스 분리 단독 +1–7%, per-bank와 결합 시 71.8 → 80.7%. refresh의 PRE · REFpb가 행 버스로 분리되어 RD 흐름 유지
- **뱅크 그룹이 최적 매핑을 바꿈**: 연속 16 word가 한 그룹에 있는 XOR은 연속 읽기 50%로 제한, 2 word마다 그룹을 전환하는 row:col:bank는 78.5%
- **경로별 latency**: hit RD 5 · miss RD 8 · conflict RD 11사이클, posted WR와 포워딩된 RD는 1사이클 (v3부터)

Traffic × 기능 Matrix 분석: 빌드 16 × Traffic 7 × 부하 2종 = 1,456회

v4 기본 대비 파라미터 단일 변경 빌드를 동일 트래픽으로 수행 · 동일 파라미터로 합성 (make matrix · ppa-sweep)



데이터 버스 사용률(%). 왼쪽 랜덤 부하(seed 10), 오른쪽 포화(seed 3). 1,456회 모두 PASS, 규칙 위반 0, 데이터 불일치 0 (RTL ab9d522, 스크립트 322632f).

분석 항목별 결론 (포화 평균 기준)

항목	결론	근거
최대 성능 요인	**per-bank refresh**	all-bank로 바꾸면 –16%, 면적 차이 2%
명령 버스 분리	면적 비용 없음	하나로 바꾸면 –7%, 면적 0.4% 이내
주소 매핑	**XOR만** 전 stride 대응	64 word stride: XOR 27.0%, 나머지 8.5–10.0%
큐 8 + 8	처리량 +6% 대비 면적 +46%	4 + 4는 읽기 latency 절반, 면적 –31%
RD 파이프라이닝	필수	RD 1개로 줄이면 –19%, 면적 0.4%
FR-FCFS vs FCFS	본 모델에서는 동등	FCFS가 면적 +8% (두 큐 간 age 필요)
쓰기 묶음	평균 ↓, tail ↑	읽기 평균 –1.7%, p99 +9% (랜덤 부하 7패턴), 방향 전환 –19%
선제 RFM	제거 가능	성능 동일, 면적 –0.8%

fmax 차이가 ABC 편차(±12%) 이내면 구조 차이로 판단하지 않음: 비트 배선만 다른 3개 매핑이 모두 310 MHz, XOR 게이트 2개가 추가된 기본 빌드가 348 MHz. FF 수는 정확, 면적은 ±2–3% 이내 신뢰 가능.

PPA: 기능 확장 대비 Fmax 유지 — Version별 면적 · Timing, 기능별 Cost

sv2v → Yosys 0.52 → Nangate45 typical → OpenSTA 3.1 · mem_ctrl 단독(Pre-layout, 클럭 트리 없음) · 버전별 해당 commit 재합성

버전별 (make ppa)

	v1 fa08330	v2 c1b8248	v3 초기 구현 미커밋	v3 1b0943a	v4 ab9d522
Area μm²	9,692	13,478	19,263	18,600	**18,912**
Cell	4,433	6,781	10,991	9,924	**9,995**
FF	1,072	1,344	1,582	1,582	**1,616**
reg2reg ns	2.880	2.753	4.765	2.922	**2.831**
≈ fmax MHz	343	357	208	337	**348**

v3 타이밍 클로저: 208 → 264 → 337 MHz

- **증상**: 쓰기 모드 → 후보 선택 → 큐 pop → 동일 주소 비교 → 포워딩 · req_ready → push. fanout 58 · 104 net 2개가 2 ns
- **① 수락 경로 분리**: 동일 주소 비교 시 이번 엣지 발행 엔트리도 포함 → 스케줄러 결정 대기 제거
- **② 후보 집합 통합**: "선택된 큐, 없으면 다른 큐" 단일 집합으로 oldest 선택 5 → 3회, pop 시 의존 비트 clear 제거
- **③ one-hot 유지**: index 인코딩 · 디코딩 단계(fanout 72) 제거, AND-OR 선택
- **v4**: 버스 분리로 RD/WR · ACT/PRE 간 선택 단계 제거 → RFM 추가 후에도 348 MHz (RFM 전 377 MHz)

기능별 제거 영향 (포화 평균 · 랜덤 부하 읽기 latency)

기능 (대체)	처리량 Δ	latency Δ	기능 면적	판단
per-bank refresh (all-bank)	−16%	+10%	+2%	최소 비용 · 최대 효과
두 명령 버스 (하나)	−7%	+4%	~0%	비용 거의 없음
RD 4개 in flight (1개)	−19%	+8%	+0.4%	필수
큐 8 + 8 (4 + 4)	−6%	−45%	+46%	처리량 우선일 때만
XOR 매핑 (row:bank:col)	stride64 −69%	+28%	편차 범위	기본 유지
FR-FCFS (FCFS)	0%	0%	−8%	본 모델에선 면적 이득
쓰기 묶음 (오래된 순)	0%	+2%	−2%	평균 · 전환 vs p99
선제 RFM (한도 도달 시만)	0%	0%	+0.8%	제거 가능

Δ는 기본 빌드 대비 해당 기능 제거 빌드의 변화, 면적은 해당 기능이 기본 빌드에 추가하는 양. fmax는 편차(±12%)를 넘는 차이만 유의(depth4 +18%, timer −23%, fcfs_nodrain −24%). 권장: 범용은 현재 기본값, 면적 · latency 우선 시 큐 4 + 4(처리량 −6%, 면적 −31%). ≈ fmax = 1 / 최소 주기(reg2reg + 플롭 setup, v4는 2.831 + 0.043 = 2.874 ns).

12 Conclusion

핵심 요약 및 향후 계획

Spec → RTL → 검증 → 성능 / PPA 분석 전 과정 결과와 미해결 항목 정리

핵심 요약

동일 DRAM 모델에서 컨트롤러를 5단계로 설계하며 전환마다 성능과 비용을 수치로 비교하고,
독립 체커와 랜덤 회귀(1,200회, 위반 0)로 단계마다 정확성을 확인

향후 계획

- 뱅크 그룹 word 단위 interleaving XOR 매핑 (연속 주소 · stride 동시 대응)
- P&R Flow(OpenROAD) 기반 기능별 타이밍 비용 재측정
- 응답 버퍼(FF 34%) 축소, dram_model 오류 경로 negative test
- DFI 1 : 2(400 MHz, 클럭당 명령 2개) 및 RD_PIPE의 PHY 왕복 지연 정합 (HSL 통합 도출 과제)
- 실제 장치 규모(pseudo channel · 뱅크 16개 이상)에서 refresh · RFM 비중 재평가

권용현

010-6408-8646

kyongh0331@naver.com

github.com/yhkwon-git/mini-dram-controller